

Code Explanation

Delta Live Tables Pipeline Code Explanation

This Delta Live Tables pipeline is designed to process customer and order data, implementing SCD Type 2 for the order summary table. The pipeline reads data from CSV files, cleans and transforms the data, and loads it into various tables.

Overview

The code is structured into multiple steps, each representing a specific task in the data processing pipeline. It starts by defining schemas for customer and order data, followed by reading the source CSV data into bronze tables. The data is then cleaned and transformed into silver tables. Finally, the code creates an SCD Type 2 order summary table and a customer aggregate spend table.

Step 1: Define Schemas for Customer and Order Data

This step defines the schemas for customer and order data using PySpark's StructType and StructField. The schemas specify the column names, data types, and whether each column is nullable.

```
customer_schema = StructType([
    StructField("CustId", StringType(), True),
    StructField("Name", StringType(), True),
    StructField("EmailId", StringType(), True),
    StructField("Region", StringType(), True)
])

order_schema = StructType([
    StructField("OrderId", StringType(), True),
    StructField("ItemName", StringType(), True),
    StructField("PricePerUnit", DoubleType(), True),
    StructField("Qty", IntegerType(), True),
    StructField("Date", DateType(), True),
    StructField("CustId", StringType(), True)
])
```

Step 2: Read Source CSV Data into Bronze Tables

This step reads the customer and order data from CSV files into bronze tables using PySpark's read function.

```
@dlt.table(
    name=f"{CUSTOMER_TABLE}_bronze",
    comment="Raw customer data from CSV files"
)
def customer_bronze():
    return (
        spark.read.format("csv")
        .option("header", "true")
        .schema(customer_schema)
        .load(CUSTOMER_DATA_PATH)
    )

@dlt.table(
    name=f"{ORDER_TABLE}_bronze",
    comment="Raw order data from CSV files"
```

```

)
def order_bronze():
    return (
        spark.read.format("csv")
        .option("header", "true")
        .schema(order_schema)
        .load(ORDER_DATA_PATH)
    )

```

Step 3: Clean Customer Data

This step cleans the customer data by removing null values and duplicates.

```

@dlt.table(
    name=CUSTOMER_TABLE,
    comment="Cleaned customer data"
)
def customer_silver():
    return (
        dlt.read(f"{CUSTOMER_TABLE}_bronze")
        .filter(
            (F.col("CustId").isNotNull()) &
            (F.col("Name").isNotNull()) &
            (F.col("EmailId").isNotNull()) &
            (F.col("Region").isNotNull())
        )
        .dropDuplicates(["CustId"])
    )

```

Step 4: Clean Order Data and Add TotalAmount Column

This step cleans the order data by removing null values and duplicates, and adds a new column TotalAmount.

```

@dlt.table(
    name=ORDER_TABLE,
    comment="Cleaned order data with TotalAmount calculated"
)
def order_silver():
    return (
        dlt.read(f"{ORDER_TABLE}_bronze")
        .filter(
            (F.col("OrderId").isNotNull()) &
            (F.col("ItemName").isNotNull()) &
            (F.col("PricePerUnit").isNotNull()) &
            (F.col("Qty").isNotNull()) &
            (F.col("Date").isNotNull()) &
            (F.col("CustId").isNotNull())
        )
        .withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qty"))
        .dropDuplicates(["OrderId"])
    )

```

Step 5: Create SCD Type 2 Order Summary Table

This step creates an SCD Type 2 order summary table by joining customer and order data, and implementing SCD Type 2 logic.

```
@dlt.table(
    name=ORDER_SUMMARY_TABLE,
    comment="Order summary with customer details as SCD Type 2",
    table_properties={
        "delta.enableChangeDataFeed": "true"
    }
)
@dlt.expect_all_or_drop({"valid_custid": "CustId IS NOT NULL"})
def order_summary():
    # Get current data in the target table if it exists
    try:
        existing_data = spark.table(f"{TARGET_CATALOG}.{TARGET_SCHEMA}.{ORDER_SUMMARY_TABLE}")
        has_existing_data = True
    except:
        has_existing_data = False

    # ... rest of the code for SCD Type 2 implementation ...
```

Step 6: Implement SCD Type 2 Logic

This step implements the SCD Type 2 logic by identifying new and changed records, expiring old records, and creating new active records.

```
# Identify new and changed records
existing_customer_attrs = existing_data.filter(F.col("IsActive") == True).select(
    "CustId", "Name", "EmailId", "Region"
).distinct()

current_customer_attrs = customer_df.select(
    "CustId", "Name", "EmailId", "Region"
).distinct()

# Find changed customer records
changed_customers = (
    current_customer_attrs
    .join(existing_customer_attrs, "CustId", "inner")
    .where(
        (current_customer_attrs["Name"] != existing_customer_attrs["Name"]) |
        (current_customer_attrs["EmailId"] != existing_customer_attrs["EmailId"]) |
        (current_customer_attrs["Region"] != existing_customer_attrs["Region"])
    )
    .select(current_customer_attrs["CustId"])
    .distinct()
)

# ... rest of the code for SCD Type 2 implementation ...
```

Step 7: Create Customer Aggregate Spend Table

This step creates a customer aggregate spend table by aggregating the total amount by customer name and date.

```
@dlt.table(
    name=CUSTOMER_AGGREGATE_SPEND_TABLE,
    comment="Aggregated customer spending by name and date"
)
def customer_aggregate_spend():
    # Read from the order summary table, only active records
    order_summary_df = dlt.read(ORDER_SUMMARY_TABLE).filter("IsActive = true")

    # Aggregate total amount by customer name and date
    return (
        order_summary_df
        .groupBy("Name", "Date")
        .agg(F.sum("TotalAmount").alias("TotalAmount"))
    )
```